

Reinforcement Learning

Clase 07

Ignacio Brevis

`ignaciobrve@unisabana.edu.co`

Universidad de La Sabana

30 septiembre 2026



Basada en los apéndices de Zhao

Parte 1: recomendada

Parte 2: opcional

Fundamentos de probabilidad

Reinforcement Learning, segundo semestre

Ignacio Brevis

(guía de estudio basada en los Apéndices A y B del libro de S. Zhao)

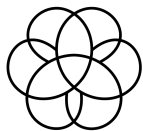
*Versión 1.0. Si encuentras algún error o tienes comentarios, escríbeme a
ignaciobreve@unisabana.edu.co.*

Tres guías disponibles:

- Sistemas lineales desde cero
- Variables aleatorias desde cero
- Probabilidades para RL

Resumen

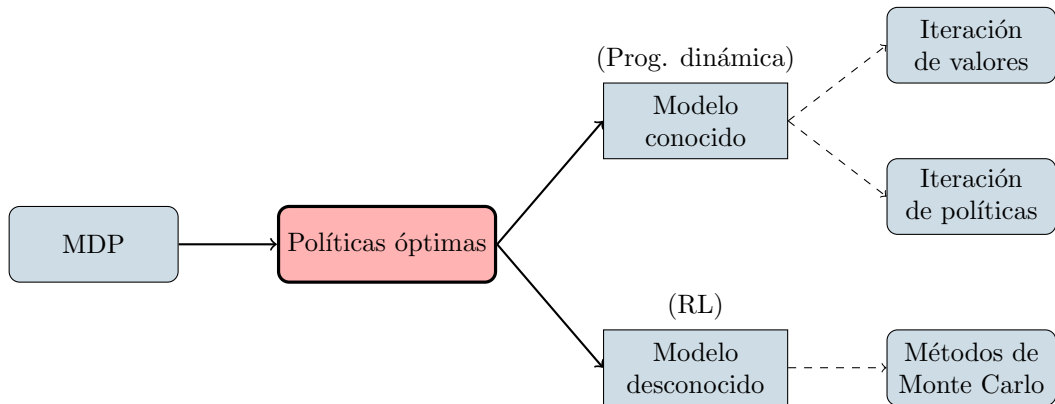
El aprendizaje por refuerzo (*reinforcement learning*, RL), en el fondo, trata sobre **estimar esperanzas** a partir de muestras. El valor de un estado es una esperanza; el gradiente de política es una esperanza; los algoritmos TD, Sarsa y Q-learning son procedimientos estocásticos cuya corrección se enuncia en términos de convergencia *casi segura*. Esta guía reúne, de forma autocontenida, el mínimo indispensable de teoría de la probabilidad para leer sin problemas un texto de RL. La Parte I cubre el material elemental (Apéndice A) y la Parte II introduce la probabilidad basada en teoría de la medida (Apéndice B), que es la que da sentido preciso a expresiones como $\mathbb{E}[X|Y]$ y a frases como «el algoritmo converge con probabilidad 1». Al final se incluyen 12 ejercicios con pistas y soluciones completas.



Gymnasium

- Gymnasium es una biblioteca de Python que ofrece entornos de RL.
- Continuación de OpenAI Gym.
- Mantenido desde 2022 por la Farama Foundation (<https://farama.org>).
- Usaremos una versión de gymnasium ≥ 1.3 .
- Pueden ver <https://github.com/ibreviis/in-deep-week-intro-RL> si necesitan ayuda con la instalación.

<https://gymnasium.farama.org/index.html>



(MC basic, MC exploring starts, MC ϵ -greedy)

Aproximación estocástica se refiere a una amplia clase de algoritmos iterativos para resolver problemas de raíces y problemas de optimización.

El algoritmo de Robbins-Monro (1951), permite resolver problemas de la forma

$$g(w) = 0$$

cuando **no conocemos g y solo tenemos acceso a observaciones con ruido** $\tilde{g}(w, \eta) := g(w) + \eta$ por medio de

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k)$$

Handwritten note: + ruido

A STOCHASTIC APPROXIMATION METHOD¹

By HERBERT ROBBINS AND SUTTON MONRO

University of North Carolina

1. Summary. Let $M(x)$ denote the expected value at level x of the response to a certain experiment. $M(x)$ is assumed to be a monotone function of x but is unknown to the experimenter, and it is desired to find the solution $x = \theta$ of the equation $M(x) = \alpha$, where α is a given constant. We give a method for making successive experiments at levels $x_1, x_2, \dots$ in such a way that x_n will tend to θ in probability.

2. Introduction. Let $M(x)$ be a given function and α a given constant such that the equation

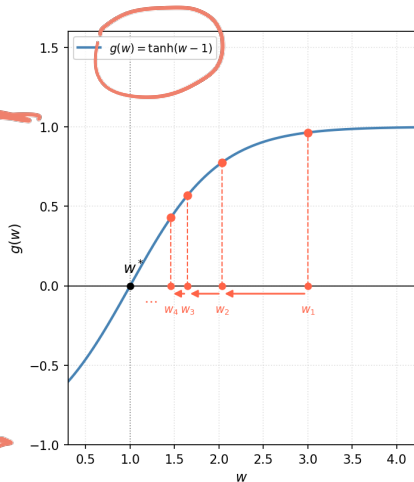
$$(1) \quad M(x) = \alpha$$

has a unique root $x = \theta$. There are many methods for determining the value of θ by successive approximation. With any such method we begin by choosing one or more values $x_1, \dots, x_r$ more or less arbitrarily, and then successively obtain new values x_n as certain functions of the previously obtained $x_1, \dots, x_{n-1}$, the values $M(x_1), \dots, M(x_{n-1})$, and possibly those of the derivatives $M'(x_1), \dots, M'(x_{n-1})$, etc. If

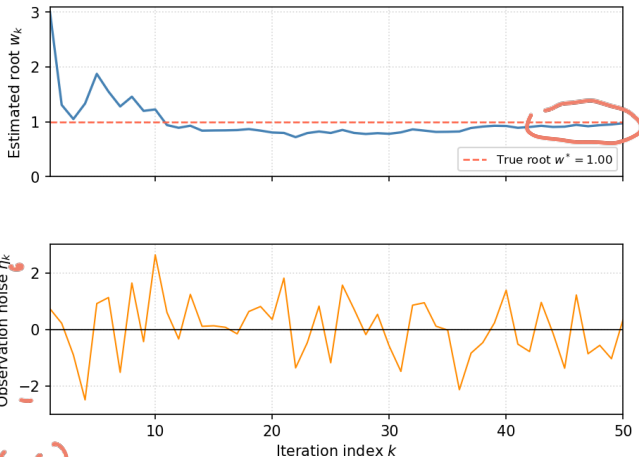
$$(2) \quad \lim_{n \rightarrow \infty} x_n = \theta,$$

H. Robbins and S. Monro, A Stochastic Approximation Method, *Ann. Math. Stat. or Ann. Math. Statist.*, 1951.

Repaso: Robbins-Monro algorithm



Illustrative example of the RM algorithm
 $g(w) = \tanh(w - 1)$, $w_1 = 3.0$, $a_k = 1/k$, $\eta_k \sim \mathcal{N}(0, 1)$



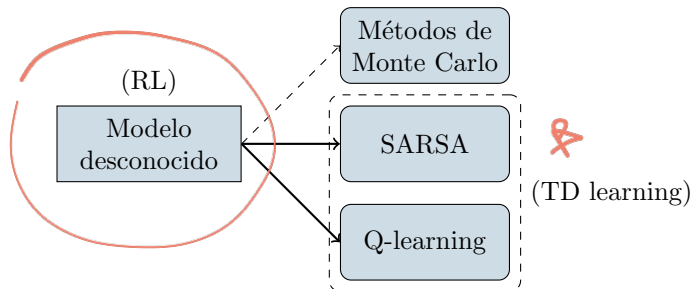
$$w = \mathbb{E}(x)$$

$$g(w) = w - \mathbb{E}(x)$$

Repaso: TD methods

Los métodos de **temporal-difference** (TD) learning combinan ideas de los métodos de Monte Carlo y de programación dinámica (DP).

- **MC: No requiere modelos** y aprende de las experiencias (requiere completar un episodio antes de calcular retornos)
- **DP:** Requiere modelo y utiliza **bootstraps** (actualiza $v(s)$ usando $v(s')$ de los estados vecinos)



Bellman equation

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s], \quad s \in \mathcal{S}$$

Bellman expectation equation

*

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s], \quad s \in \mathcal{S}$$

Robbins-Monro para Bellman expectation equation

$$g(v(s)) := v(s) - \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s]$$

$$\tilde{g}(v(s)) := v(s) - (r_{t+1} + \gamma v_{\pi}(s_{t+1}))$$

R.M.

$$v_{k+1}(s_t) := v_k(s_t) - \alpha_k \left[v_k(s_t) - (r_{t+1} + \gamma v_{\pi}(s_{t+1})) \right]$$

TD methods: Learning state values

Dada una política π , el objetivo es calcular $v_\pi(s)$ para todo $s \in \mathcal{S}$. Supongamos que tenemos muestras de algunas experiencias generadas por π :

$$(s_0, r_1, s_1, r_2, s_2, \dots, s_t, r_{t+1}, s_{t+1}, \dots)$$

Entonces, queremos estimar la **función de valor** por medio del algoritmo de temporal-difference:

$$* \left\{ \begin{array}{l} \underline{v_{t+1}(s_t)} = v_t(s_t) - \alpha(s_t) \left[v_t(s_t) - (r_{t+1} + \gamma \underline{v_t(s_{t+1})}) \right], \end{array} \right. \quad (1)$$

$$* \left\{ \begin{array}{l} \underline{v_{t+1}(s)} = \underline{v_t(s)}, \quad \forall s \neq s_t \end{array} \right. \quad (2)$$

Obs: El algoritmo de temporal-difference (como modificación de Robbins-Monro) converge a $v_\pi(s_t)$.

Obs: El término “temporal-difference learning” normalmente se refiere a una clase más amplia de algoritmos de RL.

TD methods: Learning state values

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t) \left[\underbrace{v_t(s_t) - \overbrace{(r_{t+1} + \gamma v_t(s_{t+1}))}^{\text{target } \bar{v}_t}}_{\text{error } \delta_t} \right]$$

$\downarrow$ nueva est. $\downarrow$ est. actual

$$\bar{v}_t = r_{t+1} + \gamma v_t(s_{t+1}) \quad [\text{TD target}]$$

$$\delta_t = v_t(s_t) - \bar{v}_t \quad [\text{TD error}]$$

TD methods: Learning state values

$$\underbrace{v_{t+1}(s_t)}_{\text{nueva estim.}} = \underbrace{v_t(s_t)}_{\text{estim. actual}} - \underbrace{\alpha_t(s_t)}_{\delta_t} \left[\underbrace{v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))}_{\text{TD target } \bar{v}_t} \right]$$

TD target: Notemos que:

$$\begin{aligned} v_{t+1}(s_t) &= v_t(s_t) - \alpha_t(s_t) [v_t(s_t) - \bar{v}_t] \\ v_{t+1} - \bar{v}_t &= v_t - \bar{v}_t - \alpha_t [v_t - \bar{v}_t] \\ v_{t+1} - \bar{v}_t &= (1 - \alpha_t) [v_t - \bar{v}_t] \\ |v_{t+1} - \bar{v}_t| &= \underbrace{|1 - \alpha_t|}_{\leq 1} |v_t - \bar{v}_t| \leq |v_t - \bar{v}_t| \end{aligned}$$

TD methods: Learning state values

$$\underbrace{v_{t+1}(s_t)}_{\text{nueva estim.}} = \underbrace{v_t(s_t)}_{\text{estim. actual}} - \alpha(s_t) \left[\overbrace{v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))}^{\text{TD error } \delta_t} \right]$$

$\underbrace{(r_{t+1} + \gamma v_t(s_{t+1}))}_{\text{TD target } \bar{v}_t}$

TD error: Notemos además que el error:

$$\delta_t := v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1}))$$

refleja de alguna forma la “diferencia” entre v_t y v_π . Denotemos

$$\delta_t^\pi := v_\pi(s_t) - (r_{t+1} + \gamma v_\pi(s_{t+1}))$$

$$\begin{aligned} \mathbb{E}[\delta_t^\pi | S_t = s_t] &= \mathbb{E}(v_\pi(s_t) - (r_{t+1} + \gamma v_\pi(s_{t+1})) | S_t = s_t) \\ &= v_\pi(s_t) - \mathbb{E}(r_{t+1} + \gamma v_\pi(s_{t+1}) | S_t = s_t) \\ &= 0. \end{aligned}$$

lesson 07

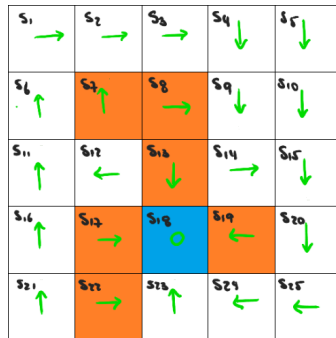
Calcular la función de valor v_π usando TD,

$$v_{t+1}(s_t) = v_t(s_t) - \alpha(s_t) \left[v_t(s_t) - (r_{t+1} + \gamma v_t(s_{t+1})) \right]$$

Ver código `Example_01_value_function_TD.ipynb` y completar la función `td_learning`

recordar que:

- Entrar a una celda blanca, $r = 0$.
- Entrar a una celda naranja, $r = -1$.
- Alcanzar la celda azul, $r = +1$.



Obs: El algoritmo presentado

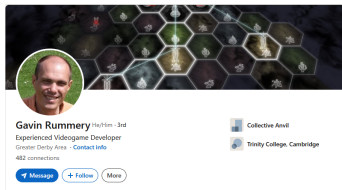
- Solamente estima la función de valor para una política dada (resuelve Bellman expectation equation)

$$\star v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s]$$

- No estima el valor de las acciones
- No calcula políticas óptimas

A continuación estudiaremos algoritmos que sí nos permiten calcular políticas óptimas.

TD methods: Learning action values (SARSA)



ON-LINE Q-LEARNING USING CONNECTIONIST SYSTEMS

G. A. Rummery & M. Niranjan

CUED/F-INFENG/TR 166

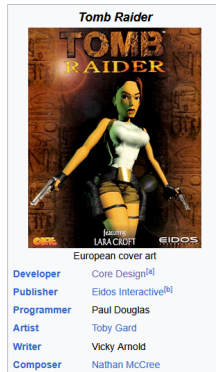
September 1994

Cambridge University Engineering Department
Trumpington Street
Cambridge CB2 1PZ
England

Email: gar@eng.cam.ac.uk, niranjan@eng.cam.ac.uk

Tomb Raider is a 1996 [action-adventure video game](#) developed by [Core Design](#) and published by [Eidos Interactive](#) as the debut entry in the *Tomb Raider* media franchise. It was first released on the [Sega Saturn](#), followed shortly by versions for [MS-DOS](#) and the [PlayStation](#). Later releases came for [Mac OS](#) (1999), [Pocket PC](#) (2002), [N-Gage](#) (2003), [iOS](#) (2013) and [Android](#) (2015). The game follows archaeologist-adventurer [Lara Croft](#), who is hired by businesswoman Jacqueline Natla to find an artefact called the Scion of [Atlantis](#). Gameplay features Lara navigating levels split into multiple areas and room complexes while fighting enemies and solving puzzles to progress.

Tomb Raider was praised for its innovative 3D graphics, controls, and gameplay. The game went on to win several industry awards and is considered to be one of the [greatest video games ever made](#). It is also one of the best-selling games for the PlayStation, with seven million units sold worldwide, and it remained the best-selling title in the *Tomb Raider* franchise until the 2013 reboot. Lara Croft herself became a cultural icon, rising to prominence as one of gaming's most recognisable characters. Following the game's success, numerous sequels were released, beginning with *Tomb Raider II* in 1997. Two remakes have been developed; *Tomb Raider: Anniversary* (2007), and the upcoming *Tomb Raider: Legacy of Atlantis* (2026). A remastered version of *Tomb Raider*, alongside *Unfinished Business*, was released as part of *Tomb Raider I–III Remastered* in 2024.



TD methods: Learning action values (SARSA)

Dada una política π , el objetivo es calcular $q_\pi(s, a)$ para todo par (s, a) . Supongamos que tenemos muestras de algunas experiencias generadas por π :

$$(s_0, a_0, r_1, s_1, a_1, \dots, s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}, \dots)$$

Entonces, queremos estimar el valor de la acción por medio del algoritmo SARSA:

$$\left\{ \begin{array}{l} q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right], \\ q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t) \end{array} \right.$$

donde $t = 0, 1, 2, \dots$

- $q_t(s_t, a_t)$ es una estimación de $q_\pi(s_t, a_t)$
- $\alpha_t(s_t, a_t)$ es el learning rate dependiendo de s_t y a_t .

TD methods: Learning action values (SARSA)

Input:

$$\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$$

S A R S A

GPI

SARSA resuelve la siguiente ecuación de Bellman:

*
$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

Obs: El algoritmo SARSA converge a $q_\pi(s_t, a_t)$.

Obs: SARSA solo puede estimar el valor de acciones (para una política dada).

Obs: SARSA, combinado con un paso de actualización de política, puede encontrar políticas óptimas (comúnmente también llamado SARSA).

TD methods: Learning action values (SARSA)

Algorithm 7.1: Optimal policy learning by Sarsa

Initialization: $\alpha_t(s, a) = \alpha > 0$ for all (s, a) and all t . $\epsilon \in (0, 1)$. Initial $q_0(s, a)$ for all (s, a) . Initial ϵ -greedy policy π_0 derived from q_0 .

Goal: Learn an optimal policy that can lead the agent to the target state from an initial state s_0 .

For each episode, do

Generate a_0 at s_0 following $\pi_0(s_0)$

If s_t ($t = 0, 1, 2, \dots$) is not the target state, do

Collect an experience sample $(r_{t+1}, s_{t+1}, a_{t+1})$ given (s_t, a_t) : generate r_{t+1}, s_{t+1} by interacting with the environment; generate a_{t+1} following $\pi_t(s_{t+1})$.

Update q -value for (s_t, a_t) :

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right]$$

Update policy for s_t :

ϵ -greedy
$$\begin{cases} \pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{if } a = \arg \max_a q_{t+1}(s_t, a) \\ \pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}(s_t)|} & \text{otherwise} \end{cases}$$

$s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$

TD learning vs MC learning

	TD learning	MC learning
	<i>online</i>	<i>offline</i>
Incremental	Sí. TD puede actualizar valores de estado/acción inmediatamente después de recibir una experiencia.	No. MC requiere que el episodio haya finalizado para calcular los retornos.
Tareas	<u>Tareas continuas y episódicas.</u>	<u>Solo tareas episódicas.</u>
Bootstrapping	Sí. TD actualiza valores a partir de estimaciones previas.	No. Estima valores calculando retornos.
Sesgo y varianza	High bias / low estimation variance	Unbiased / high estimation variance

TD methods: Learning action values (SARSA)

Calcular la política óptima π_{ϵ}^* usando SARSA,

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha(s_t) \left[q_t(s_t, a_t) - (r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})) \right]$$

Ver código:

`Example_03_optimal_policy_SARSA.ipynb`

O

`Example_02_optimal_policy_SARSA_FrozenLake.ipynb`

y completar la función `sarsa` y ajustar los parámetros de la función para obtener la política óptima.

recordar que:

- Entrar a una celda blanca, $r = 0$.
- Entrar a una celda naranja, $r = -1$.
- Alcanzar la celda azul, $r = +1$.

s_1	s_2	s_3	s_4	s_5
s_6	s_7	s_8	s_9	s_{10}
s_{11}	s_{12}	s_{13}	s_{14}	s_{15}
s_{16}	s_{17}	s_{18}	s_{19}	s_{20}
s_{21}	s_{22}	s_{23}	s_{24}	s_{25}

Image adapted from Zhao, Mathematical Foundations of Reinforcement Learning.